

Events Extraction from Political-theme News

Qui Hung Kieu^{1,2}, Nguyen Phuc Mac^{1,3}, and Duy Liem Nguyen^{1,4}

¹ Faculty of Information Science and Engineering,
University of Information and Technology

² 22520505@gm.uit.edu.vn

³ 22521123@gm.uit.edu.vn

⁴ 22520752@gm.uit.edu.vn

Abstract. Event extraction is a core task in information extraction, playing a crucial role in transforming unstructured textual data into structured, machine-readable information. In this study, we propose a deep learning-based system for extracting events from Vietnamese political news articles, collected from the online platform **baochinhpvu.vn**. Two main approaches are developed and evaluated: (1) a traditional *pipeline model*, in which event trigger detection and argument extraction are treated as separate sequential stages; and (2) a *joint learning model* inspired by OneIE, which simultaneously learns multiple tasks to enhance contextual understanding and overall performance. To encode the semantic representation of input texts, we employ XLM-RoBERTa, a powerful multilingual pretrained transformer model. Experimental results demonstrate that the joint learning approach significantly outperforms the pipeline method, achieving higher macro F1 scores across all sub-tasks, including event type classification, trigger tagging, and argument role identification. Furthermore, the system is integrated with Apache Kafka and Spark Structured Streaming to enable real-time event extraction from continuous streams of political news. The proposed solution is scalable and efficient, offering a promising tool for automatic event monitoring and political trend analysis in the Vietnamese language. This framework can support downstream applications such as crisis alerting and policy tracking.

Keywords: Events Extraction · Political News · BaoChinhPhu · Trigger Tagging · Event Type Classification · Arguments Detection.

1 INTRODUCTION

In the era of explosive digital information, the volume of unstructured textual data originating from online news outlets, social media platforms, and mass media is increasing at an unprecedented pace. Among these, the political domain—characterized by its timeliness and profound societal impact—has become a focal point for monitoring, analysis, and trend forecasting. However, manually extracting knowledge from political news is both time-consuming and labor-intensive, often prone to human error. This reality underscores the urgent need for automated

systems capable of understanding and extracting structured information from text.

Event Extraction (EE) is a core task in semantic information mining that aims to identify real-world events mentioned in text. It typically involves three subtasks: detecting event triggers, classifying event types, and identifying participant entities (arguments) along with their semantic roles. This is a challenging problem that requires deep contextual understanding and the ability to process multiple layers of linguistic information simultaneously. The challenge becomes even greater in Vietnamese—a low-resource language—due to the scarcity of annotated datasets and robust natural language processing tools.

In this study, we aim to develop an automated event extraction system tailored to Vietnamese political news articles, leveraging state-of-the-art deep learning techniques. Specifically, we implement and compare two main approaches: (1) a pipeline-based model, which processes trigger detection, argument extraction, and event classification sequentially; and (2) a joint learning model, inspired by the OneIE architecture, which integrates all three tasks into a unified framework. Both approaches utilize XLM-RoBERTa, a powerful multilingual pretrained transformer, to enhance semantic representation capabilities.

Beyond model design, our system is integrated into a real-time data processing pipeline using Apache Kafka and Spark Structured Streaming. This enables event extraction from continuously generated streams of political news, offering timely insights.

The goal of this project is not only to improve the accuracy of event recognition but also to build a comprehensive and scalable system that can support downstream applications such as political crisis monitoring, policy analysis, and strategic media planning.

2 RELATED WORKS

Event Extraction (EE) is a critical subfield of Information Extraction (IE) that has garnered considerable attention from the research community in recent years. Traditional approaches often rely on rule-based systems, domain-specific lexicons, or supervised models with handcrafted features [1]. While these methods have shown effectiveness in certain domains, they generally lack generalizability and are difficult to adapt to low-resource languages such as Vietnamese.

The rise of deep learning techniques—particularly convolutional neural networks (CNNs) and recurrent neural networks (RNNs)—has led to significant improvements in EE performance [2]. However, these models still exhibit limitations in capturing long-range dependencies and tend to treat subtasks such as trigger detection, argument extraction, and event classification as independent modules, potentially missing valuable cross-task interactions.

A major advancement in the field came with the introduction of Transformer-based architectures, most notably BERT [3] and its multilingual variants like XLM-RoBERTa [4], which offer enhanced contextualized semantic representations and support for multiple languages, including Vietnamese. Building on this

foundation, recent studies have increasingly adopted joint learning architectures to train multiple EE tasks simultaneously, promoting richer interaction among linguistic layers. A representative example is OneIE [5], a comprehensive joint learning system that achieved state-of-the-art results on standard benchmarks such as ACE 2005 and Rich ERE.

In the Vietnamese context, EE research remains relatively limited, primarily due to the scarcity of annotated datasets and robust natural language processing tools. Notable efforts include VietnameseEEN [6], which focuses on dataset construction and evaluating Vietnamese BERT models for event extraction. However, these studies have not explored real-time applications or integration with big data platforms such as Kafka and Spark.

Our work builds upon and extends these prior directions by applying a joint learning architecture with XLM-RoBERTa as the backbone for Vietnamese. Furthermore, we integrate the system into a real-time processing pipeline using Apache Kafka and Spark Structured Streaming, enabling event extraction from continuous streams of political news.

3 DATA CONSTRUCTION

Data plays a pivotal role in training and evaluating deep learning models for the Event Extraction (EE) task. Although several benchmark datasets for Vietnamese have been developed in the past, many of them are now outdated—particularly in the political news domain, where information evolves daily and requires continuous updates. To address this gap, we constructed a custom dataset tailored specifically for Vietnamese political news. This dataset serves as the foundation for model training, testing, and evaluation in our system. To enhance the reliability of annotated data in the semi-automatic labeling process, we employed Cohen’s Kappa coefficient to measure inter-annotator agreement. On a randomly sampled 10% subset of the dataset (comprising 100 instances), we achieved an agreement score of 73.9% for event extraction and 75.1% for event argument extraction—indicating substantial consistency among annotators. Following this, the remaining data within this subset was manually labeled individually to complete the annotation phase. Our dataset construction approach aims to strike a balance between annotation quality and scalability, ensuring the resulting corpus is both representative of real-world political discourse and suitable for training robust deep learning models.

3.1 Data Collecting from baochinhphu.vn

One of the primary data sources utilized in this study is the Politics section of the official government news portal, accessible at <https://baochinhphu.vn>. The objective of the data acquisition process was to construct a dataset of politically related news articles, each annotated with structured fields including the title, URL, publication date, and content summary.

The data collection pipeline was designed in two main phases. In the first phase, we employed the `requests` library to send HTTP requests to individual pages within the Politics section. The returned HTML content was then parsed using `BeautifulSoup`. Specific HTML tags containing article information were identified based on characteristic classes such as `box-category-item`, `list__main`, and `box-stream-item`. Article titles and relative URLs were extracted and normalized into complete URLs. To avoid duplicate entries, we used a `set` to store the collected URLs, and excluded the first two articles on each subsequent page to eliminate overlaps. The resulting list of articles was saved in a CSV file for further preprocessing.

In the second phase, the program iterated through the previously collected URLs to access the detailed content of each article. From these pages, the publication date and summary were extracted using specific HTML elements, such as `div` tags with classes `"time"` or `"summary"`. The final result was consolidated and stored in a structured table format in the file `demo.csv`.

This pipeline enabled us to collect over 5,000 high-quality political articles from a reliable source, ensuring consistency and accuracy in the input data. The collected corpus served as the foundation for the subsequent annotation and model training stages in our event extraction system.

3.2 Data Cleaning

Following the data collection phase from political news sources, we conducted a preprocessing step to ensure the integrity, consistency, and quality of the input data for model training. Initially, all article URLs were standardized and checked for duplication. Duplicate links—whether occurring across different pages or within the same page—were removed. Additionally, invalid entries, such as articles missing a title or URL, were excluded from the dataset to maintain completeness.

For the summary content, we performed text cleaning by removing unnecessary prefixes such as `“(Chinhphu.vn) -”` to ensure that the summaries accurately reflected the article content. The *publish date* and *summary* were extracted directly from each detailed article page. All cleaned and normalized data were then stored in a structured CSV file format, serving as the foundation for subsequent annotation and model training procedures.

3.3 Prompting Guideline

In this study, we employ a prompting-based approach to automate the annotation process for political event extraction from Vietnamese text. The guideline is designed to reduce manual effort while ensuring annotation consistency across the dataset. The main steps are as follows:

1. **Data Preprocessing:** Raw texts are cleaned and normalized to improve quality before being processed by the language model. Typical preprocessing steps include removing special characters, converting text to lowercase,

sentence segmentation, and correcting spelling or formatting issues when necessary.

2. **Prompt Generation:** For each sentence or paragraph, the system generates task-specific prompts designed to guide the language model toward extracting relevant information. Examples of such prompts include:
 - *"Identify any political events mentioned in the following text:"*
 - *"Does this paragraph contain any political events? If so, please list them."*
3. **Sending Prompts to a Language Model:** The generated prompts are sent to a large language model (LLM), such as GPT or similar, to obtain predicted annotations for each input segment.
4. **Post-processing of Model Output:** The responses from the language model are parsed and converted into a structured format (e.g., JSON or tabular form). These outputs are then used to label the original text accordingly.
5. **Evaluation and Refinement:** The auto-generated labels are reviewed, either manually or via rule-based validation mechanisms, to ensure annotation quality. Based on the results, the prompts or pipeline components may be refined to improve accuracy and reliability.

This prompting-based guideline significantly accelerates the data annotation process, minimizes manual labor, and promotes consistency across the dataset, making it well-suited for large-scale event extraction tasks.

3.4 Labeling data using a combination of Rule-based and LLM.

To construct a labeled dataset for training our event extraction model, we adopted a hybrid annotation strategy that combines two techniques: **rule-based semi-automatic labeling** and **automatic labeling using a large language model (LLM)**. This combined approach leverages the strengths of both methods—rule-based techniques provide high precision for well-defined patterns, while LLMs offer broader coverage for more flexible or complex linguistic cases.

1. **Semi-automatic labeling using rule-based methods:** We first applied handcrafted rules based on keywords, regular expressions, and semantic templates to automatically annotate key fields such as `event_type`, `trigger`, and `argument`. This was implemented through a custom function, `extract_event_info`, and the output was stored in an intermediate `DataFrame` (e.g., `final_df`).
2. **Identifying unannotated entries:** After the rule-based step, we filtered the dataset to identify entries where the `event_type` or related fields remained empty (NaN or blank strings). These entries were flagged for further processing via an LLM.
3. **Automatic labeling using LLM:** For the remaining unlabeled entries, we employed a large language model (e.g., GPT-4) to extract event information from the summary content (`summary`). Each entry was transformed into a prompt with a predefined format and submitted to the LLM. The extracted outputs were parsed and inserted back into the corresponding fields of the `DataFrame`.

4. **Merging and saving final results:** Finally, we combined the results from both the rule-based and LLM-based steps into a unified dataset, which was saved as a complete CSV file (e.g., `final_output_labeled.csv`) for use in subsequent model training phases.

This approach optimizes both annotation efficiency and coverage: clear-cut cases are quickly and accurately processed via deterministic rules, while more nuanced instances are addressed through the advanced language understanding capabilities of LLMs. The selective delegation of tasks also minimizes the cost of LLM usage while preserving overall data consistency.

3.5 Preprocessing Data

During the preprocessing phase prior to annotation and model training, we applied a series of steps to ensure the integrity and consistency of the input data.

First, we filtered out any entries missing essential information, such as empty `tokens` or corresponding BIO labels. In particular, we removed entries where the lengths of the `tokens` and `bio_tags` fields did not match, to preserve alignment and structural correctness.

Next, all string-based fields were parsed into actual Python lists using `ast.literal_eval`, allowing for straightforward manipulation during training. The BIO tagging schema was standardized by constructing a `label_to_id` mapping table, which converts textual labels into numerical identifiers—ensuring consistency across the entire dataset.

Finally, the dataset was split into three subsets: training, development (dev), and test. This was performed using a *stratified multilabel split* strategy to guarantee that all significant labels were represented in each split. This prevents label sparsity in any subset and enhances the generalization capacity of the trained models. An additional verification step was included to report label frequencies and flag any missing labels in the dev or test sets.

This rigorous preprocessing pipeline produces a clean, consistent, and label-diverse dataset that effectively supports deep learning models for the event extraction task.

3.6 Data Statistics

After the annotation process, the final dataset consists of approximately **5,814 sentences** with over **10,234 event triggers** and **25,819 event arguments**. The dataset is divided into three subsets: training (80%), testing (10%), and validation (10%). Basic statistics are summarized in Table1.

Figure1 illustrates the distribution of data across different event types after the annotation process. It is evident that the dataset is highly imbalanced.

- **Meeting** events dominate the dataset, with over 175 samples, indicating that meetings and official working sessions between authorities or leaders are the most frequently reported events in political news.

Table 1: Statistics of the Vietnamese Political Event Extraction Dataset

Component	Quantity
Number of sentences	5,814
Number of triggers	10,234
Number of arguments	25,819
Number of event types	12
Training set	4,000 sentences
Test set	500 sentences
Validation set	500 sentences

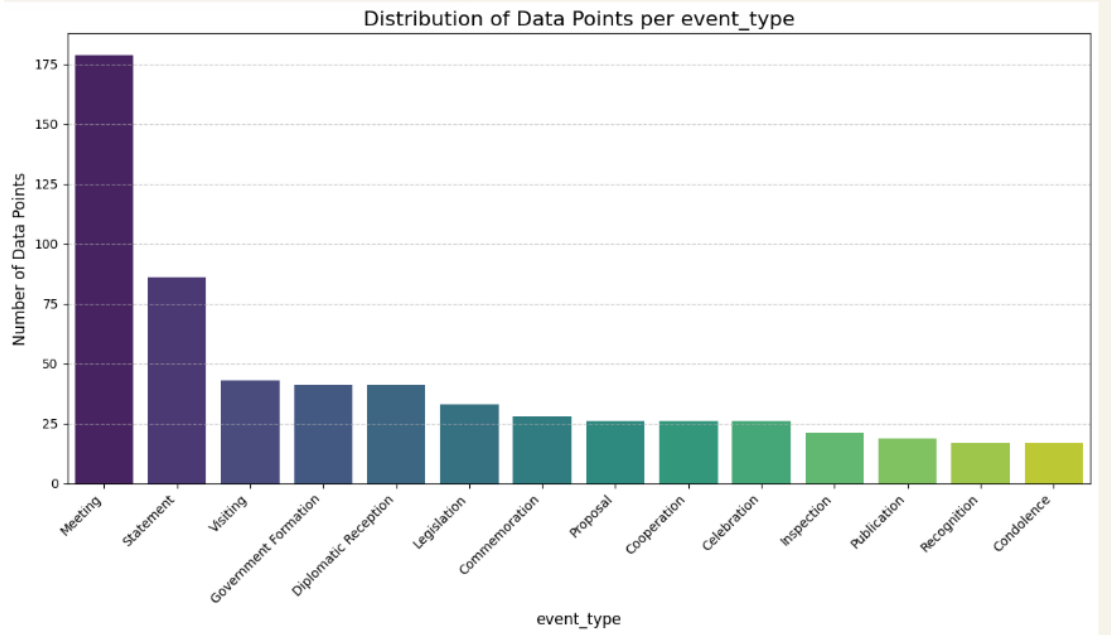


Fig. 1: Distribution of event types in the annotated dataset.

- **Statement** events also appear frequently, with approximately 85 samples, reflecting the prevalence of official announcements in political discourse.
- Other event types such as **Visiting**, **Government Formation**, **Diplomatic Reception**, and **Legislation** appear with relatively even frequency, each ranging around 40 samples.
- In contrast, event types like **Condolence**, **Recognition**, **Publication**, and **Inspection** are significantly underrepresented, with only around 20 instances each. This reflects their specific and less frequent occurrence in collected articles.

The label imbalance is a critical factor to consider during model training, as it may lead to biased learning and reduced overall performance. Consequently,

techniques such as class-weighted loss functions or data sampling strategies should be considered to mitigate the impact of underrepresented classes and enhance model generalization.

4 METHODOLOGY

The task of event extraction from Vietnamese political texts comprises three main subtasks:

- **Event type classification** – identifying the type of event being described in the sentence;
- **Trigger detection** – locating the word or phrase that triggers the event;
- **Argument extraction** – identifying the entities involved in the event, such as the subject, object, time, and location.

The objective is to develop a unified model capable of performing all three subtasks simultaneously for a given input sentence. By doing so, the system can transform unstructured textual information into structured representations, enabling downstream tasks such as automatic monitoring, analysis, and decision support in the political domain and the process based on Figure 2

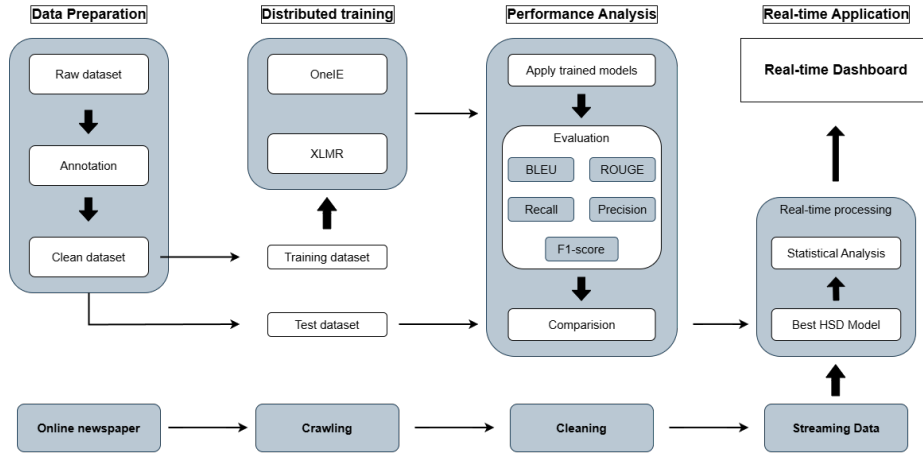


Fig. 2: Implementation Process.

4.1 Modeling Approaches

During the development of our event extraction system, we explored and implemented two main approaches to address the subtasks, while leveraging the powerful semantic representation capabilities of pre-trained Transformer models.

The selection of such models is motivated by their proven effectiveness in various NLP tasks, particularly for low-resource languages like Vietnamese. Specifically, the two approaches are as follows:

1. **Pipeline Approach** In this approach, the complex event extraction task is decomposed into a sequence of independent subtasks: *event type classification*, *trigger tagging*, and *event argument extraction*. Each subtask is handled by a dedicated model trained and optimized for its specific objective. The output of one model in the sequence is passed as input to the next, forming a pipeline of sequential processing.
 - **Applied Model:** In our study, we employed the multilingual **XLM-RoBERTa (base version)** as the core encoder for each subtask in the pipeline. XLM-RoBERTa is a robust pre-trained model trained on massive multilingual corpora, making it well-suited for Vietnamese NLP tasks such as text classification and sequence labeling. *The hyperparameters include: learning rate = $2e-5$, batch size = 8, number of epochs = 10, weight decay = 0.01, early stopping = 3, evaluation every 100 steps, and model checkpointing based on macro F1 score.*
2. **Joint-Learning Approach** Contrary to the pipeline approach, the joint-learning method aims to address all subtasks of event extraction (event type classification, trigger detection, and argument extraction) simultaneously using a single unified model. The key motivation behind this approach is to minimize error propagation—where errors from earlier stages can negatively impact subsequent stages in a pipeline—and to capture the interdependence between the subtasks. This allows the model to learn shared representations and potentially achieve higher overall performance.
 - **Applied Model:** We implemented a joint learning model inspired by the architecture of **OneIE**. Specifically, the model utilizes the strong encoder backbone of XLM-RoBERTa and simultaneously performs the three core tasks via three parallel classifier branches operating on the encoder’s output. Although our implementation does not include advanced features of the original OneIE (e.g., event graph networks, beam search decoding, or token pair interactions), due to the absence of the required annotations in our dataset, our model retains the essential end-to-end joint learning characteristics. This enables the system to leverage global context and optimize extraction performance effectively.

4.2 Implementation Methods

To operationalize the event extraction process, we decomposed the overall task into three core subtasks. Each of these plays a vital role in transforming unstructured textual data into structured event representations. The implementation strategies for each subtask are as follows:

Event Type Classification This subtask focuses on identifying the specific category of event described in a given sentence. As the first and foundational step in the event extraction pipeline, accurate event type classification plays a crucial role in guiding subsequent extraction processes. We approach this as a text classification problem, in which a model is trained to analyze contextual cues and relevant keywords to assign appropriate labels such as *Meeting*, *Statement*, *Diplomatic Reception*, and others. The classifier is trained on a labeled dataset where each sentence is annotated with a clearly defined event type, enabling the model to learn fine-grained distinctions between different political event categories.

Trigger Tagging Trigger detection involves identifying the key word or phrase (referred to as the *trigger*) in a sentence that most explicitly evokes or expresses the occurrence of an event. Triggers are typically verbs or action-related nouns that serve as the semantic nucleus of the event. For example, in the sentence “The President **held talks** with the foreign leader,” the word *held talks* functions as the trigger. We treat this task as a sequence labeling problem, in which each token in the sentence is assigned a label indicating whether it is a trigger and, if so, what event type it corresponds to. This requires the model to capture both local and global contextual information in order to accurately identify event indicators within varying linguistic structures.

Event Arguments Extraction Following the identification of the event type and trigger, the next subtask is to extract relevant *arguments*—entities and roles associated with the event instance. These arguments provide detailed information about the event, including its participants (e.g., agent, patient), temporal expressions, and location mentions.

For example, in the event “held talks,” the arguments may include “the President” (agent), “the foreign leader” (recipient), “May 10” (time), and “in Hanoi” (location).

This subtask is commonly implemented using Named Entity Recognition (NER) techniques combined with Semantic Role Labeling (SRL), or alternatively, through relation-based extraction methods. The goal is not only to detect relevant entities but also to assign them the correct semantic roles in the context of the previously identified event, enabling a structured and complete representation of the event instance.

5 APACHE KAFKA

Apache Kafka is a distributed real-time event streaming platform that operates based on a publish-subscribe architecture. It enables applications to send and receive message-based data in a decoupled and scalable manner, facilitating seamless communication across system components: **Producer:** Sends messages to Kafka topics, **Topic:** Acts as a logical channel for categorizing and storing

messages by subject, **Consumer**: Subscribes to topics to retrieve and process messages in real-time; can operate independently or as part of a consumer group, **Zookeeper (or Kafka Raft)**: Handles cluster coordination, metadata management, and broker monitoring to ensure system consistency and fault tolerance.

6 EXPERIMENTS AND RESULTS

6.1 Evaluation Metrics

To assess the performance of the models in the event extraction task, we adopt a set of widely used evaluation metrics in the field of Natural Language Processing. These metrics provide a multi-faceted view of the model’s ability to detect and extract information accurately, including both the quality of generated texts (if applicable) and the correctness of classification outputs.

Given that event extraction is a multi-class and multi-label task with potentially imbalanced data distributions across event types (as observed in the Dataset Statistics section), we particularly emphasize the use of **macro-averaged metrics**. Macro-averaging computes Precision, Recall, and F1-score independently for each class and then takes the unweighted average across classes. This ensures that all event types are equally represented in the final evaluation, preventing dominant classes from overshadowing the minority ones and providing a more faithful measure of overall model effectiveness.

1. BLEU (Bilingual Evaluation Understudy)

BLEU measures the similarity between a text generated by the model (candidate) and one or more reference texts. It is based on n -gram overlaps and is widely used for text generation tasks such as summarization or machine translation. A higher BLEU score indicates better fluency and semantic similarity. The BLEU score is typically computed as follows:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

where BP is the brevity penalty, p_n is the precision of n -grams, and w_n is the weight of each n -gram order (usually $1/N$).

2. ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE evaluates content overlap between the generated text and the reference text by counting overlapping n -grams. It is particularly useful for assessing summarization systems, ensuring that key information from the source text is preserved. Variants include ROUGE-N (n -gram overlap), ROUGE-L (Longest Common Subsequence), and ROUGE-S (Skip-gram overlap). For example, ROUGE-N is calculated as:

$$\text{ROUGE-N} = \frac{\text{Number of Overlapping } n\text{-grams}}{\text{Total } n\text{-grams in reference}}$$

3. Precision (Macro-averaged)

Precision for a given class is the proportion of correctly predicted instances (True Positives) among all instances that the model predicted as belonging to that class (True Positives + False Positives). High precision indicates a low false-positive rate.

4. Recall (Macro-averaged)

Recall (or sensitivity) for a class is the proportion of actual instances of that class that were correctly identified by the model (True Positives) out of all actual instances (True Positives + False Negatives). High recall means the model successfully captures most relevant cases.

5. F1-score (Macro-averaged)

The F1-score for a class is the harmonic mean of Precision and Recall. It is a balanced measure especially useful in imbalanced datasets, providing a single metric that accounts for both false positives and false negatives.

6.2 Prompting Results (Gemini 2.5 Flash)

During the data construction phase, we utilized the Gemini 2.5 Flash model to automatically label the majority of the dataset (90%). Evaluating the quality of this prompting-based labeling process is essential to ensure the accuracy and reliability of the training data. The evaluation results below highlight the performance of Gemini 2.5 Flash in extracting political-themed events.

F1-score Evaluation The Gemini 2.5 Flash model achieved high F1-scores for common and semantically well-defined event types such as *Meeting*, *Visiting*, and *Condolence*, as shown in Table 2. This indicates the model’s strong ability to identify events with clear contextual and lexical cues. However, its performance dropped on less frequent or more semantically subtle events like *Statement* and *Recognition*. While the model typically showed high recall (coverage), its precision was relatively lower in several categories, suggesting a tendency to overpredict or confuse similar event types.

Confusion Matrix Analysis The confusion matrix in Figure 3 provides a detailed view of model misclassifications. The results show that Gemini 2.5 Flash performed especially well on frequent and contextually clear event types, such as *Meeting* (111 out of 174 correctly predicted), *Diplomatic Reception* (37/38), and *Government Formation* (37/38). This affirms the effectiveness of LLM-based annotation when supported by sufficient and unambiguous data. However, the model tended to confuse semantically similar event types, including *Statement*, *Publication*, and *Recognition*, a common issue in event extraction due to lexical overlap and ambiguity.

Argument and Trigger Extraction Performance

Table 2: Classification performance across event types

Event type	Precision	Recall	F1-score	Support
Celebration	0.75	0.52	0.62	23
Commemoration	0.69	0.80	0.74	25
Condolence	1.00	1.00	1.00	14
Cooperation	0.74	0.61	0.67	23
Diplomatic Reception	0.69	0.97	0.80	38
Government Formation	0.66	0.97	0.79	38
Inspection	0.57	0.89	0.70	18
Legislation	0.53	0.87	0.66	30
Meeting	0.81	0.64	0.71	174
Proposal	0.51	0.83	0.63	23
Publication	0.54	0.93	0.68	15
Recognition	0.53	0.71	0.61	14
Statement	0.84	0.39	0.53	82
Visiting	0.77	0.68	0.72	40

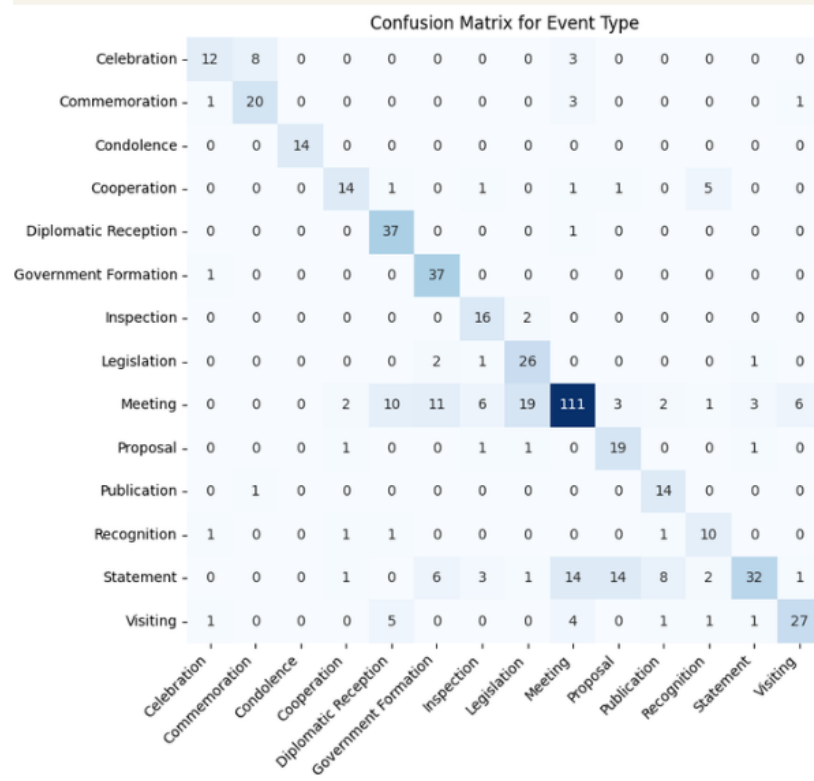


Fig. 3: Confusion matrix of predicted event labels.

- **Argument Extraction:** The model achieved high ROUGE scores (ranging from 0.82 to 0.91) when extracting structured arguments such as `arg_subject`, `arg_time`, and `arg_location`, as shown in Table3. These results suggest strong capability in identifying well-defined entities like named individuals, specific dates, and geographic locations.
- **Trigger Extraction:** In contrast, trigger extraction yielded significantly lower performance. Triggers are often verbs or nouns representing actions and require deep contextual understanding beyond simple keyword matching.
- **BLEU Score for Trigger:** The BLEU score for trigger prediction was relatively low (0.192), indicating lexical variations between predicted and reference triggers, even if their semantic meanings were aligned. This reflects BLEU’s sensitivity to exact word matching, penalizing minor discrepancies in phrasing.

Table 3: Evaluation of event component extraction using BLEU and ROUGE

Component	Avg BLEU	ROUGE-1	ROUGE-2	ROUGE-L
Trigger	0.192	0.65	0.573	0.646
Arg_Subject	0.678	0.847	0.814	0.843
Arg_Time	0.275	0.906	0.652	0.904
Arg_Location	0.331	0.826	0.386	0.823

Overall, the prompting-based annotation results with Gemini 2.5 Flash produced a high-quality and large-scale labeled dataset. This dataset serves as a solid foundation for training advanced event extraction models in the next stages of our study.

7 ERROR ANALYSIS

Although the joint learning model outperformed the pipeline approach in terms of overall performance, our evaluation revealed several recurring types of errors in the event extraction process. These errors are categorized as follows:

- **Trigger Detection Errors:** The model often fails to detect triggers in simple sentences and incorrectly identifies non-trigger words in more complex structures. Triggers in Vietnamese can be highly ambiguous and context-dependent. For example, the word “*tiếp*” may indicate a *Meeting* event in one context but not in another.
- **Argument Identification Errors:** The model frequently mislabels argument roles, particularly in sentences with inverted word order. These errors are partly due to inconsistencies introduced during automatic labeling, especially when gold-standard annotations were previously created manually. The model struggles to distinguish between agents and affected entities when the context is unclear.

- **Event Type Classification Errors:** Confusion occurs between semantically similar event types, such as *Statement* and *Meeting*. The model also exhibits poor performance on rare event types like *Condolence* and *Recognition*, primarily due to insufficient training data.
- **Data Inconsistency Errors:** Some sentences contain multiple events, but only a subset was labeled. Labeling inconsistencies between annotators were also observed, such as different trigger choices for the same event type.
- **LLM Labeling Errors:** The large language model occasionally generates hallucinated content—i.e., information not present in the input text—which leads to inaccurate labels. Some errors stem from misinterpreting ambiguous or lengthy sentences. Additionally, unclear prompting instructions caused the LLM to produce inconsistent labels across different runs.
- **Model-Specific Extraction Errors:** Errors can also be attributed to model limitations, particularly underfitting on low-resource event classes. Pipeline-based models are especially prone to error propagation, where a mistake in one step (e.g., trigger detection) compromises subsequent tasks. While joint learning alleviates this issue to some extent, it still struggles to jointly optimize all three tasks effectively when the data is noisy or incomplete.

8 CONCLUSION AND PROPOSAL

Through extensive experimentation, the event extraction system demonstrated considerable effectiveness under both approaches: Pipeline and Joint Learning (OneIE-like). Specifically, the Pipeline method achieved superior performance in the Trigger Tagging and Argument Extraction tasks, while the results for Event Type Classification were relatively similar across the two methods. Nevertheless, the joint learning approach—modeled after OneIE—showed clear advantages in terms of deployment, due to its ability to simultaneously handle all three tasks within a unified model. This not only reduces error propagation between stages but also offers computational efficiency and facilitates integration into real-time processing systems such as Kafka and Spark.

Proposed Improvements:

- **Enhancing Argument Role Learning:** Accurate identification of argument roles remains one of the most challenging aspects of event extraction. This could be improved by incorporating multi-head self-attention mechanisms or span-based architectures, which have proven effective in modeling the syntactic and semantic relationships within sentences.
- **Addressing Label Imbalance:** To boost performance for low-frequency event types, techniques such as data augmentation (e.g., constituent permutation, paraphrasing) or loss functions tailored to imbalanced data—like class-weighted loss or focal loss—should be considered to emphasize harder or underrepresented examples.
- **Extending the Label Schema:** Beyond the current label types (trigger, argument, event type), the schema can be expanded to include more fine-grained attributes such as `Arg_Object`, `Arg_Location`, or semantic relations

between arguments (e.g., **Relation**). This would not only diversify the dataset but also allow the OneIE-style model to capture deeper interdependencies within events.

- **Data Normalization and Quality Control:** A thorough review and standardization of existing labels are essential to ensure data consistency. Semi-automated validation tools or auxiliary models could be employed to assist in detecting and correcting inconsistent annotations.
- **Improving Label Quality via LLMs:** Instead of relying solely on free large language models (LLMs) such as Gemini 2.5 Flash—which may have performance limitations—investment in more powerful models like GPT-4 Turbo or Claude Opus via official APIs is recommended. Additionally, crafting clearer prompts, incorporating few-shot examples, and post-processing generated outputs are critical for enhancing label quality when using LLMs.
- **Adopting More Advanced SOTA Models:** Beyond XLM-RoBERTa base, adopting larger pre-trained models such as **PhoBERT-large** or **XLM-RoBERTa-large** could significantly enhance semantic representation, particularly for Vietnamese texts. Furthermore, newer architectures like **FourIE**—which aim for unified information extraction—present promising directions for future research and experimentation.

References

1. Grishman, R.: Event identification in ACE. *Proceedings of the Conference on Natural Language Processing* (2005)
2. Nguyen, T.H., Grishman, R.: Joint event extraction via recurrent neural networks. In: *Proceedings of NAACL* (2016)
3. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805* (2018)
4. Conneau, A., et al.: Unsupervised cross-lingual representation learning at scale. *arXiv preprint arXiv:1911.02116* (2019)
5. Lin, B.Y., et al.: A joint neural model for information extraction with global features. In: *Proceedings of ACL* (2020)
6. Nguyen, A., Tran, H.: Vietnamese event extraction: A new dataset and joint modeling approach. In: *Proceedings of the 2021 Conference of the Association for Computational Linguistics* (2021)